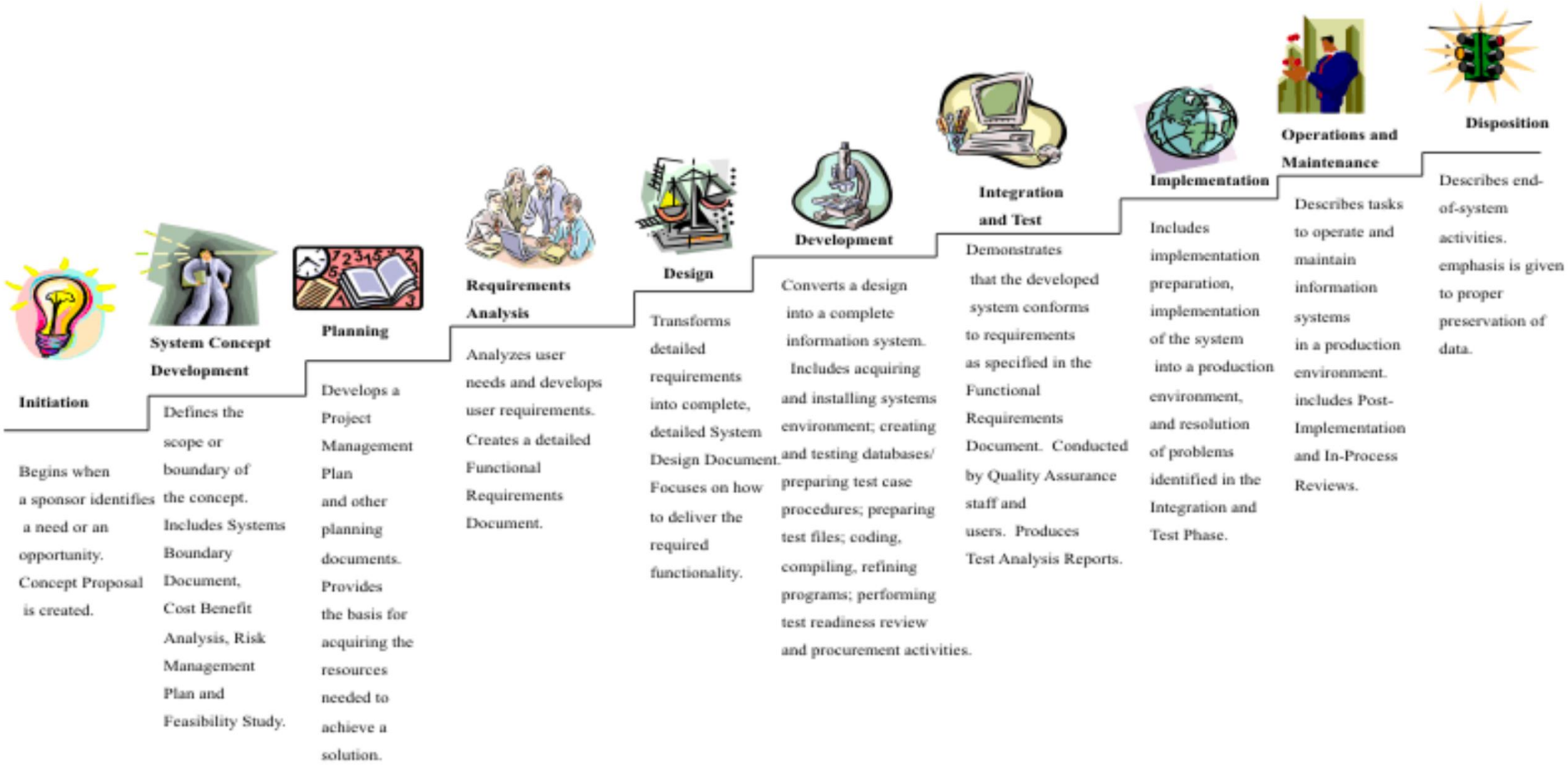
CSC 710 Software Engineering

Instructor: Dr. Tianxiao Zhang

Software Development Phases



Software Development Phases

Often the phases are summarized as follows:

- **Requirements** – determine what the client wants
- **Analysis** - determine the needs or conditions necessary to meet the new or altered product, taking into account possibly conflicting requirements from the various stakeholders
- **Design** – design the code
- **Implementation** – construct the code, test it, and install it on the client's computer

Software Development in Theory

In an ideal world, a software product is developed in phases as described theoretically:

– Developed from scratch

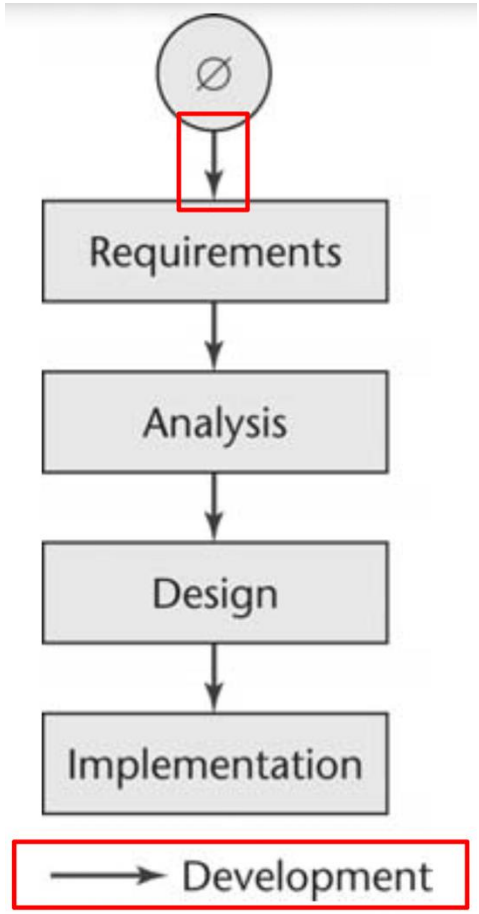
– First the client's requirements are determined

– Next, analysis focuses on determining the needs or conditions necessary to meet the new or altered product, taking into account possibly conflicting requirements from the various stakeholders.

– When analysis is complete, the design is produced

– This is followed by the implementation of the complete product

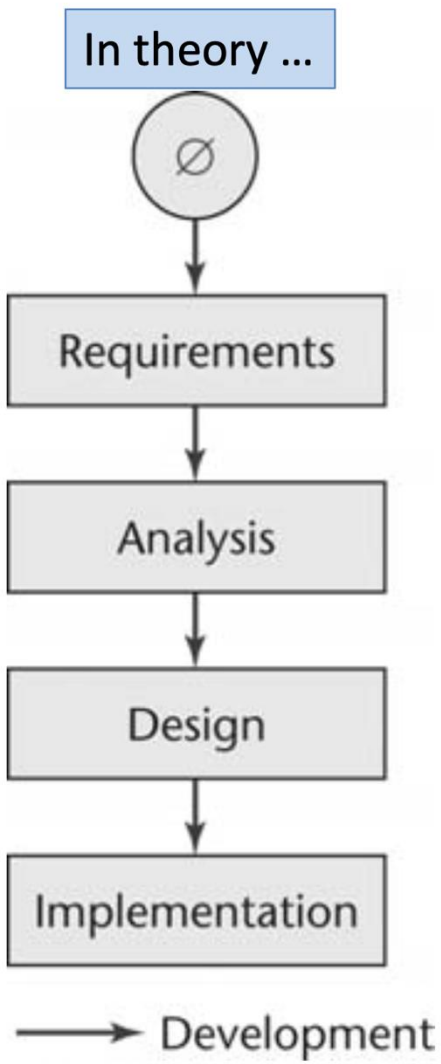
– It is then installed on the client's computer

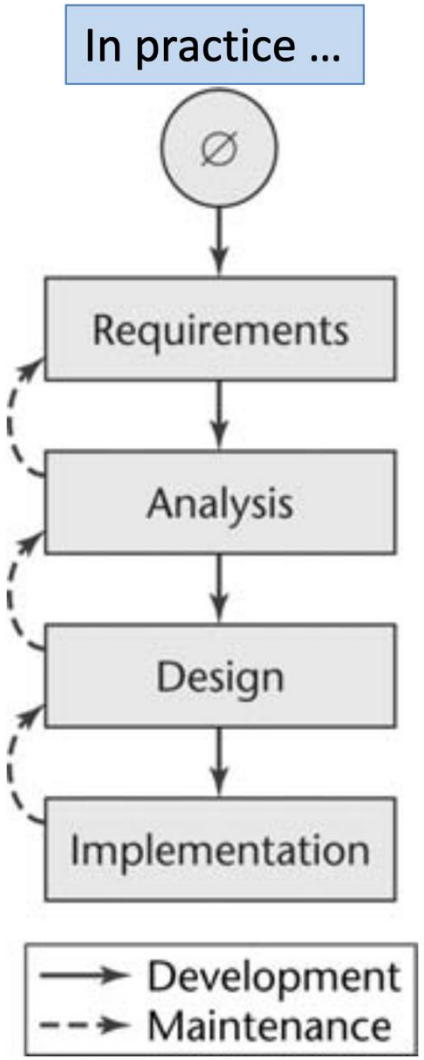


Software Development in Practice

- Software development is considerably different in practice:
  - Software professionals are human and therefore make mistakes
  - The client's requirements can change while the software is being developed
    - A product manager's changing mind regarding the functionality
    - Feature creep = Succession of small additions to requirements
- Why are so many changes to a software product needed
  - Software professionals make mistakes
  - A software product is a model of the real world, and the real world is continually changing

Software Development





Iteration & Incrementation

- The basic process of software development is iterative
  - First version produced
  - Revised to produce second
  - And so on
- Iteration is an intrinsic aspect of software engineering
- Iterative models used for over 40 years
- Another aspect of developing real-world software is the restriction imposed on us by Miller's Law
  - At any one time, humans are capable of concentrating on only approximately seven chunks (units of information)

Iteration & Incrementation

- A typical software artifact has far more than seven chunks
  - Code artifacts with more than seven variables
  - Requirements documents with many more than seven requirements
- One way humans handle this restriction is to use stepwise refinement
  - Concentrate on aspects currently most important
  - Postpone those aspects that are currently less critical
- For example, first considering the seven most important requirements, and then the next few
- This is an incremental process
- Incrementation is also an intrinsic aspect of software engineering
- Contrary to software development theory, software has been developed this way since the first code ever written.
- And this is not a bad thing because other engineering disciplines develop products in the same way.
  - Think about all the attempts humans made to fly

Strengths of Iteration & Incrementation

1. Multiple opportunities for checking that the software product is correct
   – Every iteration incorporates the test workflow
2. The robustness of the underlying architecture can be determined relatively early in the development of software
   – Architecture of a software product consists of various component artifacts and how they fit together
   – Extended continually to incorporate the next increment
   – Robustness — Being able to handle such extensions and changes without falling apart
3. Enables the early mitigation of risks
   – Risks are invariably involved in software development and maintenance
   – Risks mitigated early during incremental development

Strengths of Iteration & Incrementation

4. There is always a working version of the software

– At the end of each iteration, there is a working version of part of the overall software product

– Client and intended users can experiment with that version

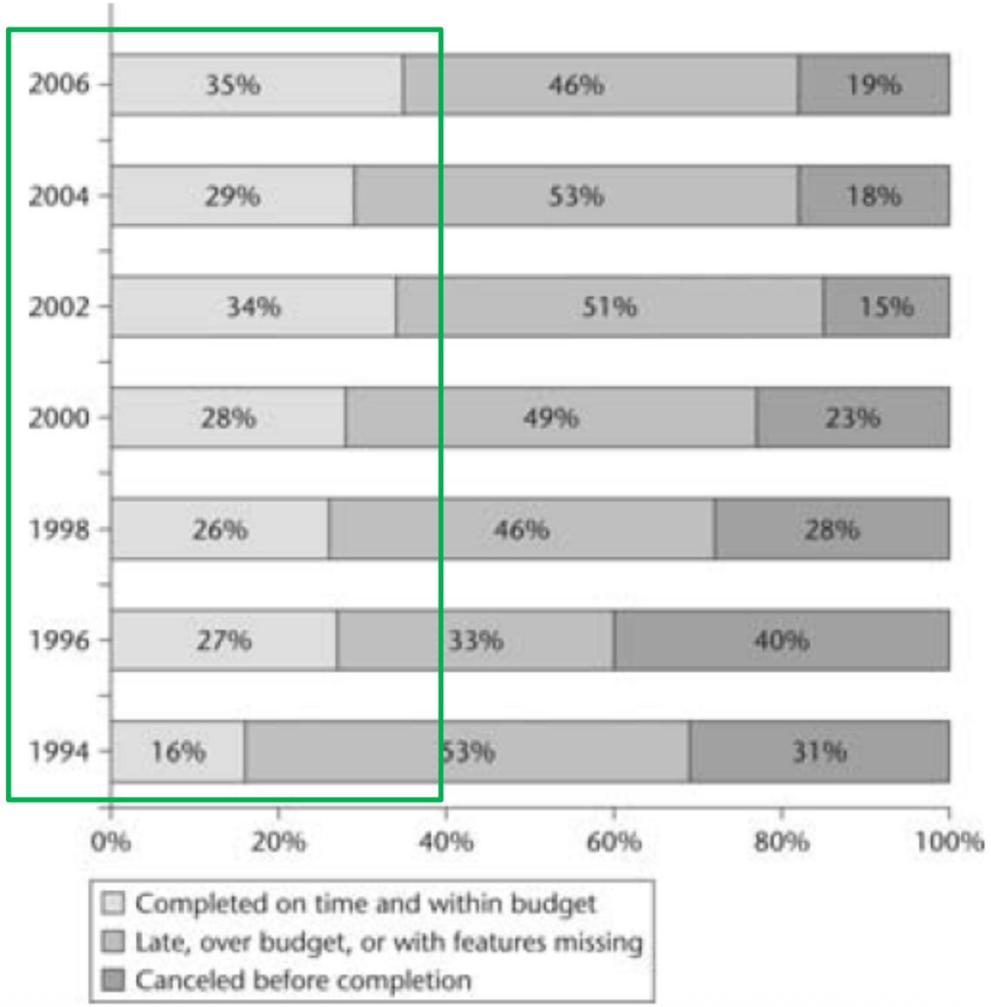
5. Benefits of introducing a software product gradually:

– Understandable fear of being replaced by a computer is diminished

– It is easier to learn the functionality of a complex product if it is introduced stepwise

Empirical Evidence that Iteration & Incrementation Works

Gradual increase in successful projects (16% to 35%) despite more large projects because of:
– Iterative development
– Better project management
– More user involvement
– More support from senior executives



Iteration & Incrementation Works … to a point

TRADITIONAL RESOLUTION FOR ALL PROJECTS

	2011	2012	2013	2014	2015
SUCCESSFUL	39%	37%	41%	36%	36%
CHALLENGED	39%	46%	40%	47%	45%
FAILED	22%	17%	19%	17%	19%

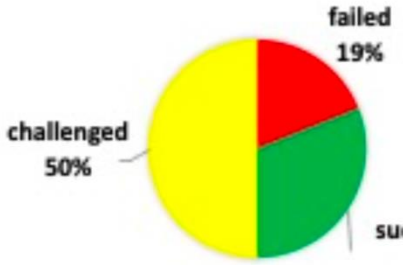
The Traditional resolution of all software projects from FY2011–2015 within the new CHAOS database.

*All data, unless otherwise noted, represents results from FY2011-2015. The total number of software projects is 25,000-plus, with an average of 5,000 per yearly period.

Iteration & Incrementation Works … to a point

Project Success
Quick Reference Card

Based on CHAOS 2020: Beyond Infinity Overview. January'2021, QRC by Henny Portman



Modern measurement
(software projects)

Good Sponsor, Good Team, and Good Place are the only things we need to improve and build on to improve project performance.

The Good Place is where the sponsor and team work to create the product. It's made up of the people who support both sponsor and team. These people can be helpful or destructive. It's imperative that the organization work to improve their skills if a project is to succeed. This area is the hardest to mitigate, since each project is touched by so many people. Principles for a Good Place are:

- The Decision Latency Principle
- The Emotional Maturity Principle
- The Communication Principle
- The User Involvement Principle
- The Five Deadly Sins Principle
- The Negotiation Principle
- The Competency Principle
- The Optimization Principle
- The Rapid Execution Principle
- The Enterprise Architecture Principle

Successful project Resolution by Good Place Maturity Level:

highly mature	50%
mature	34%
moderately mature	23%
not mature	23%

The Good Team is the project's workhorse. They do the heavy lifting. The sponsor breathes life into the project, but the team takes that breath and uses it to create a viable product that the organization can use and from which it derives value. Since we recommend small teams, this is the second easiest area to improve. Principles for a Good Team are:

- The Influential Principle
- The Mindfulness Principle
- The Five Deadly Sins Principle
- The Problem-Solver Principle
- The Communication Principle
- The Acceptance Principle
- The Respectfulness Principle
- The Confrontationist Principle
- The Civility Principle
- The Driven Principle

Successful project Resolution by Good Team Maturity Level:

highly mature	66%
mature	46%
moderately mature	21%
not mature	1%

The Good Sponsor is the soul of the project. The sponsor breathes life into a project, and without the sponsor there is no project. Improving the skills of the project sponsor is the number-one factor of success – and also the easiest to improve upon, since each project has only one. Principles for a Good Sponsor are:

- The Decision Latency principle
- The Vision Principle
- The Work Smart Principle
- The Daydream Principle
- The Influence Principle
- The Passionate Principle
- The People Principle
- The Tension Principle
- The Torque Principle
- The Progress Principle

Successful project Resolution by Good Sponsor Maturity Level:

highly mature	67%
mature	33%
moderately mature	21%
not mature	18%

Software Development Life Cycle

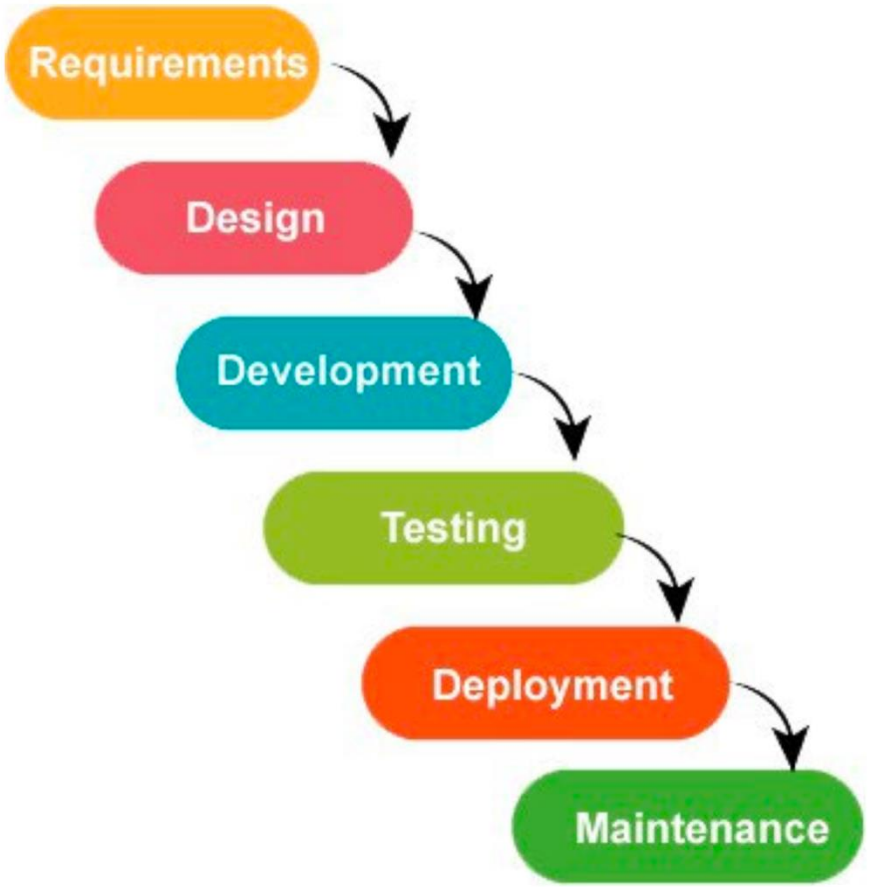
- What is a Software Development Life Cycle (SDLC) Model?

- Series of steps, or phases, to be performed while the software product is developed and maintained.

Waterfall SDLC

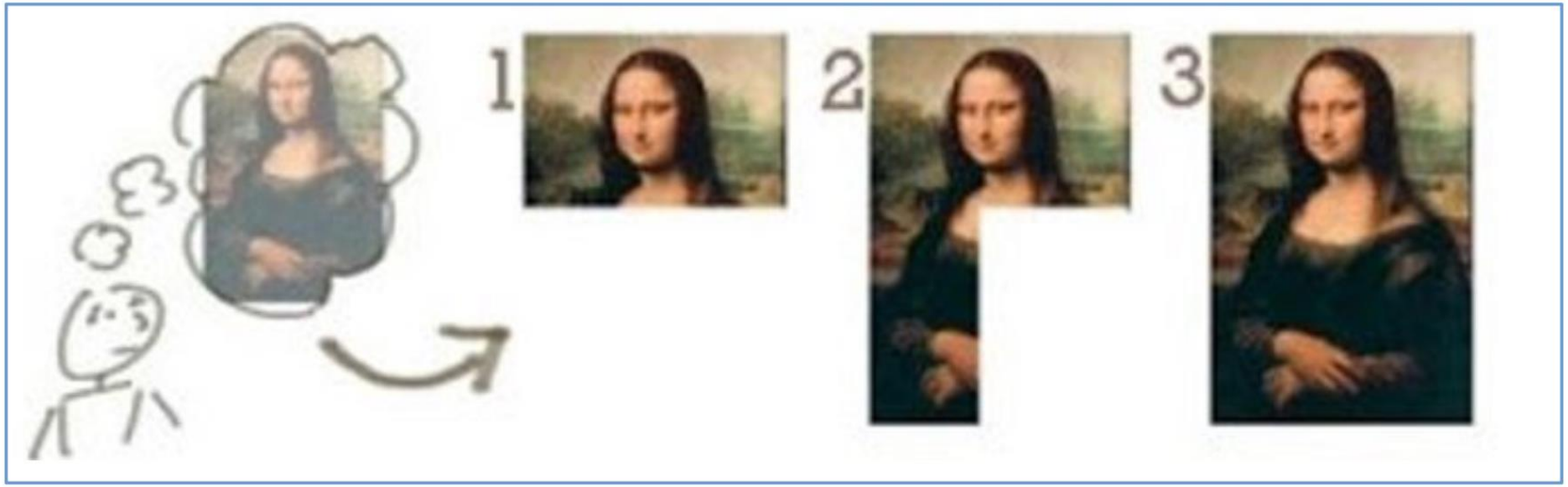
One of the first SDLC models proposed

- Each major phase is marked by milestones and deliverables (artifacts)
- Long wait before a final product
- Still used in some companies (e.g., banks and insurance companies)



Iteration & Incrementation

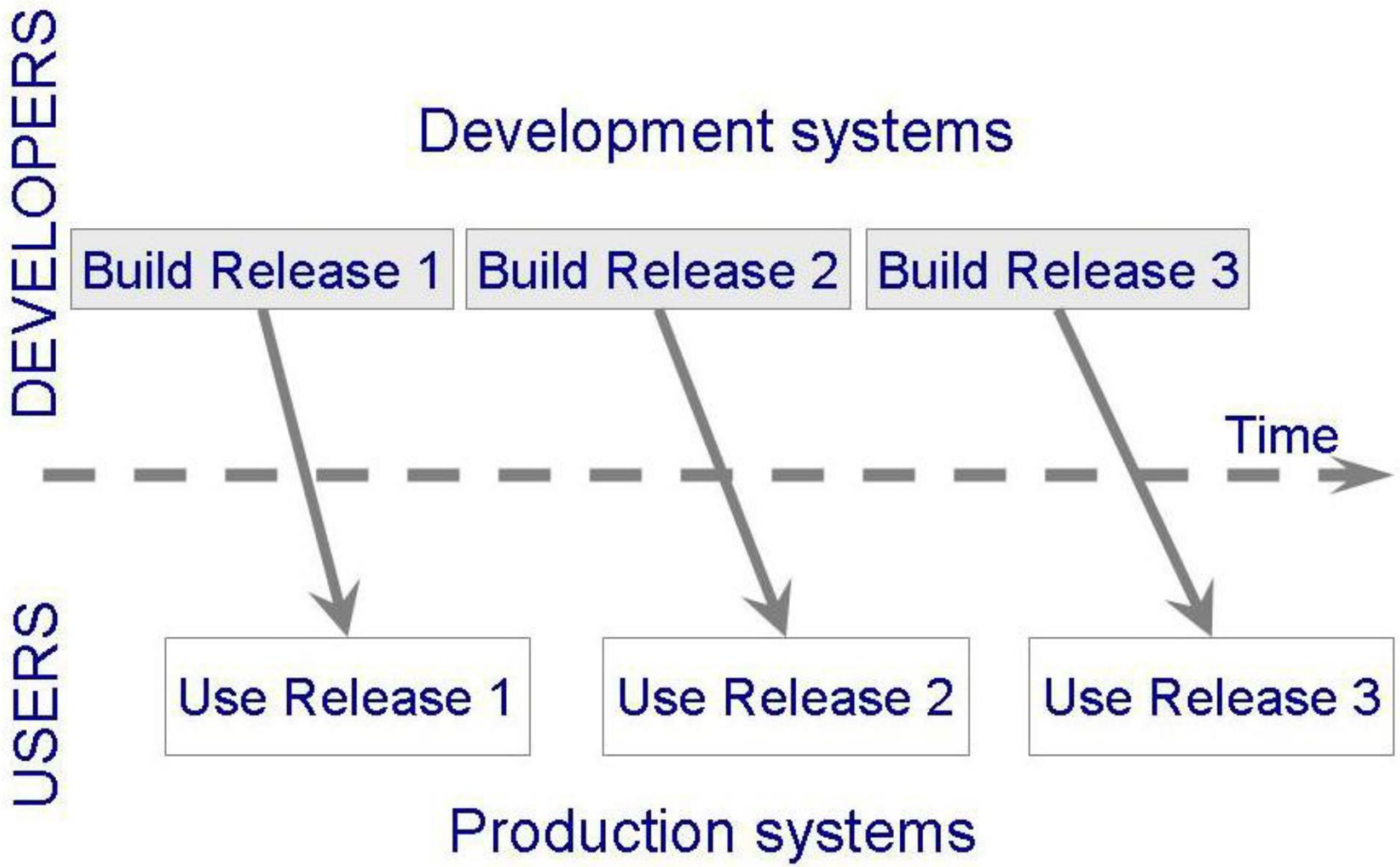
- Two ways to think about Iteration & Incrementation
- Start with small functionality and add new functionality with each new release



Iteration & Incrementation

- Start with full system (but very minimal detail) then enhance the functionality of each software component with each new iteration

Phased development: increments

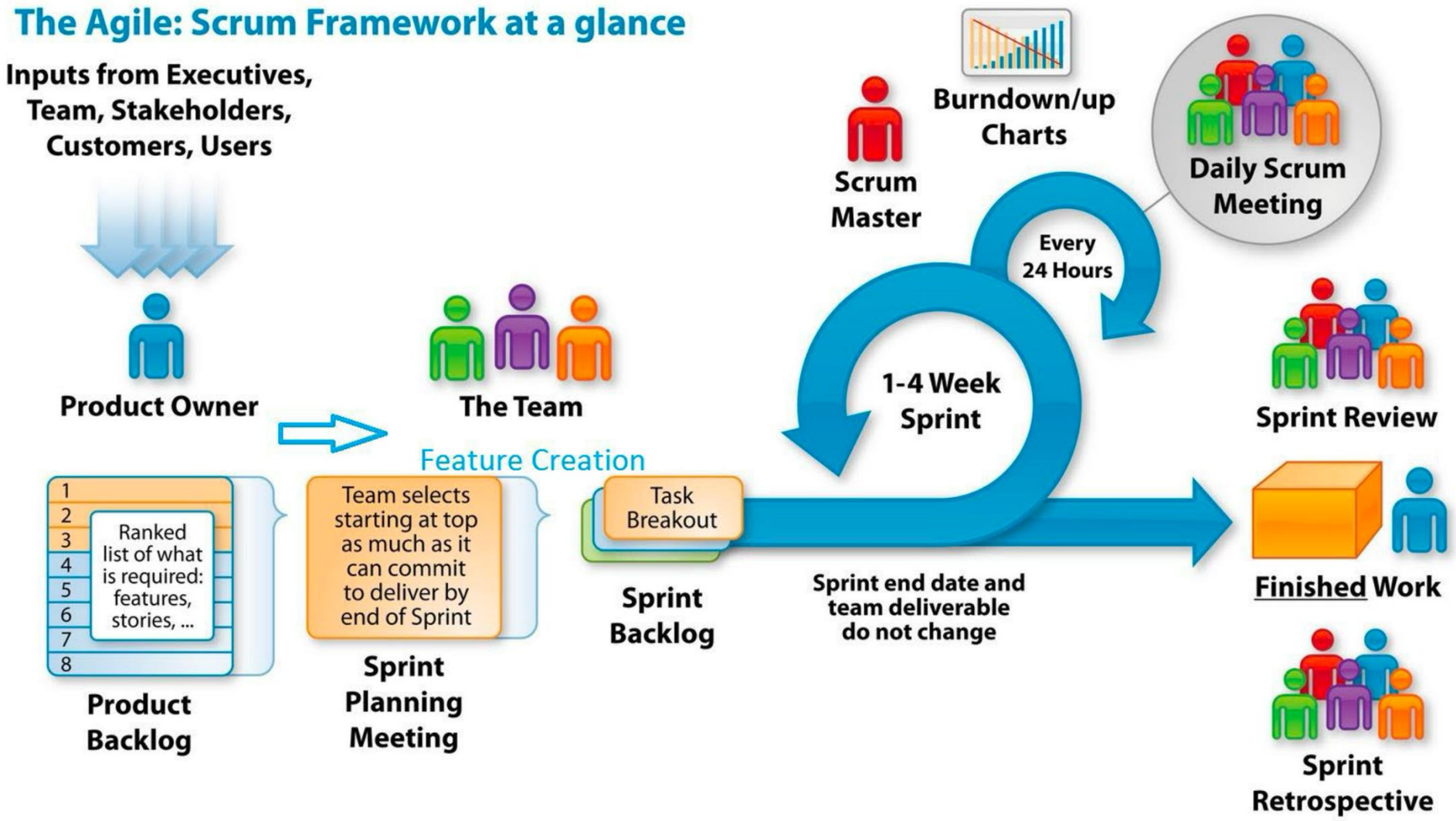


Agile SDLC

- Agile methodologies, such as Scrum and DAD, are the formalized process of Iteration & Incrementation

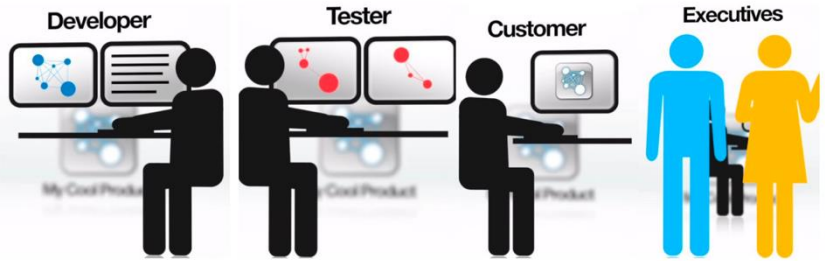
- They prioritize:
  - Flexibility
  - Iterative development
  - Stakeholder feedback
  - Lean software development (eliminating waste and optimizing processes)

Agile Scrum


The Agile: Scrum Framework at a glance
Inputs from Executives, Team, Stakeholders, Customers, Users
Product Owner
The Team
Feature Creation
1
2
3
4
5
6
7
8
Ranked list of what is required: features, stories, ...
Product Backlog
Team selects starting at top as much as it can commit to deliver by end of Sprint
Sprint Planning Meeting
Task Breakout
Sprint Backlog
Sprint end date and team deliverable do not change
1-4 Week Sprint
Every 24 Hours
Scrum Master
Burndown/up Charts
Daily Scrum Meeting
Sprint Review
Finished Work
Sprint Retrospective

Scrum Roles

- Product owner
  – Makes sure the right features get into a new release (representing the users, customers)
  – Sets direction
- Scrum master (project manager)
  – Ensures the project progresses smoothly
  – Team members have the tools to get their job done
  – Sets up meetings, monitors, plans releases, …
- Developers, testers, customers, …

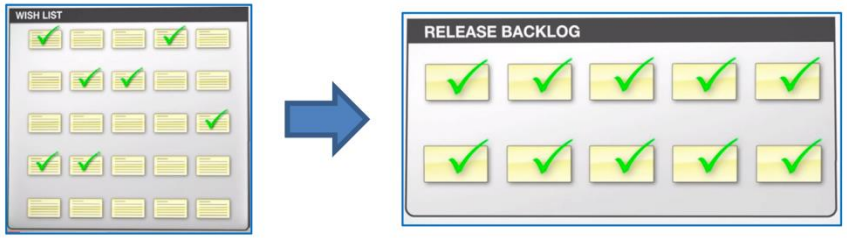


Developing Your Cool New App

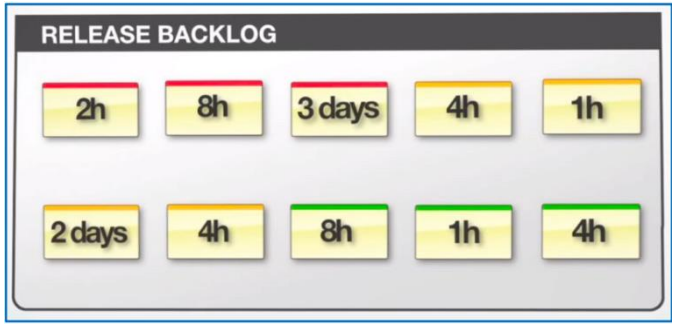
- You will get feature requests from a variety of stakeholders
  - Features are always from the perspective of the end users
  - Known as user stories: “As a user (role) I want 2F authentication”
  - The role could be a customer, an end users, an executive, …
  - Backlog: A collection of user stories

Release Planning

- Start with the user stories and create a release backlog



- Prioritize and estimate time for each

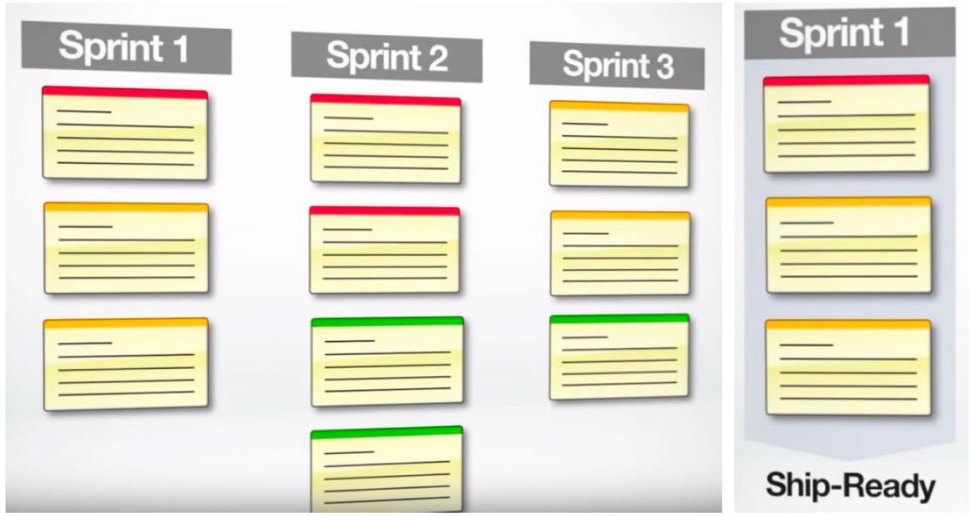


- Larger ones may be broken down

Sprints

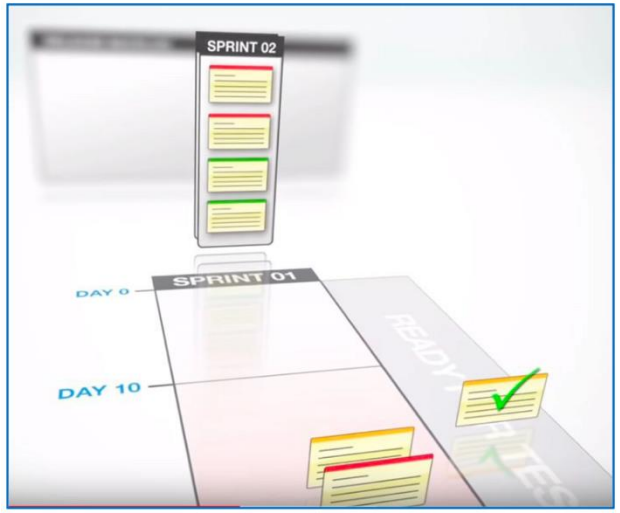
Start with the release backlog

May need several sprints to get the everything done
– Sprints: short duration milestones
– 2-30 days (depending on release cycles)

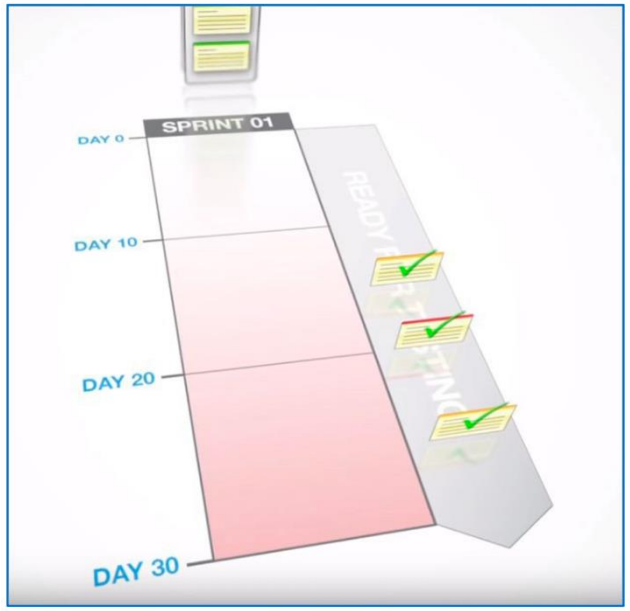


Sprint Goal

To get a subset of a release backlog to a ship-ready state



At the end of each sprint, you have a fully tested product with all features



Daily Scrum

- Daily stand up team member meetings
- To facilitate free flow of information between the team members
- Team members list:
- What they did the previous week toward the team's goals.
- What they intend to do this week toward the team's goals.
- Any roadblocks or impediments they can see to the team's goals.

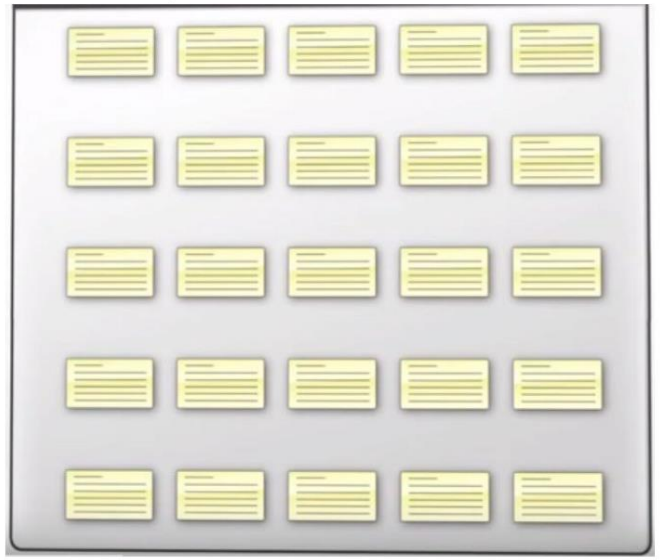
Sprint Retrospective Meeting

- At the end of each sprint, it is important to have a retrospective about what went right (to repeat) and what went bad (to avoid)

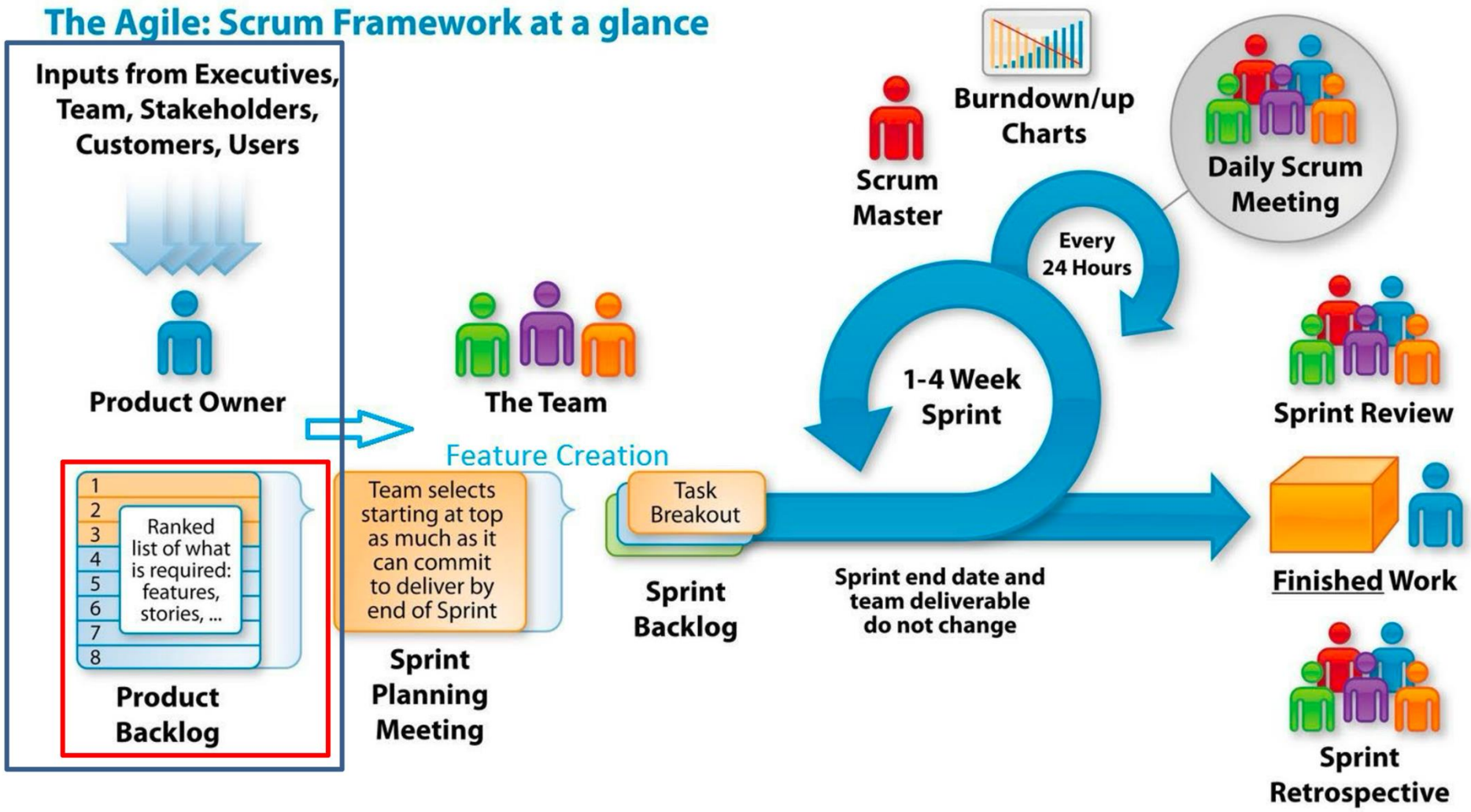
What Are Requirements?

- Requirements are what go in the "Requirements Stack" or “Product Backlog”.
- All that is necessary to define a requirement is enough words that the team knows what it means.

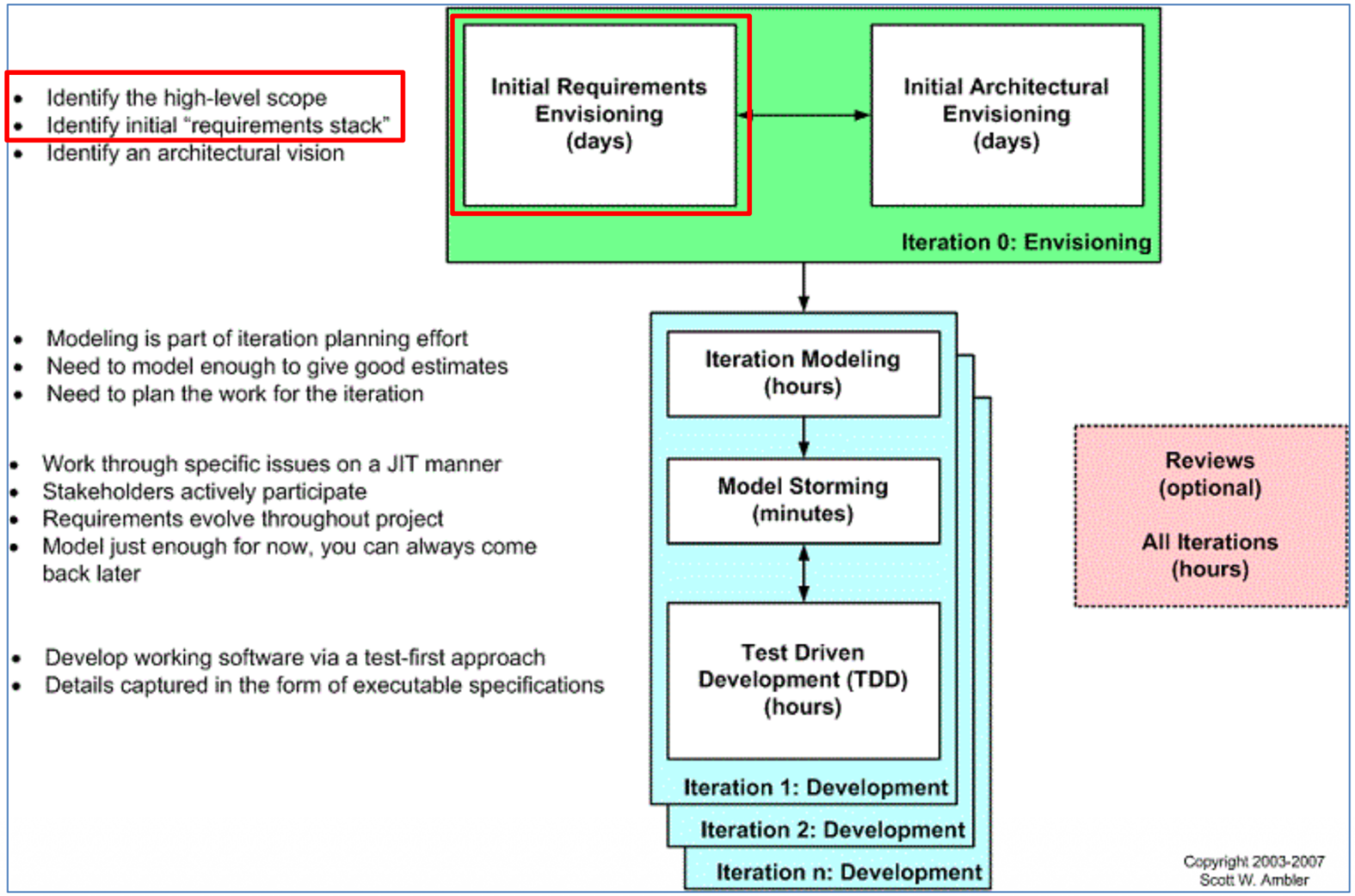
Requirements Stack



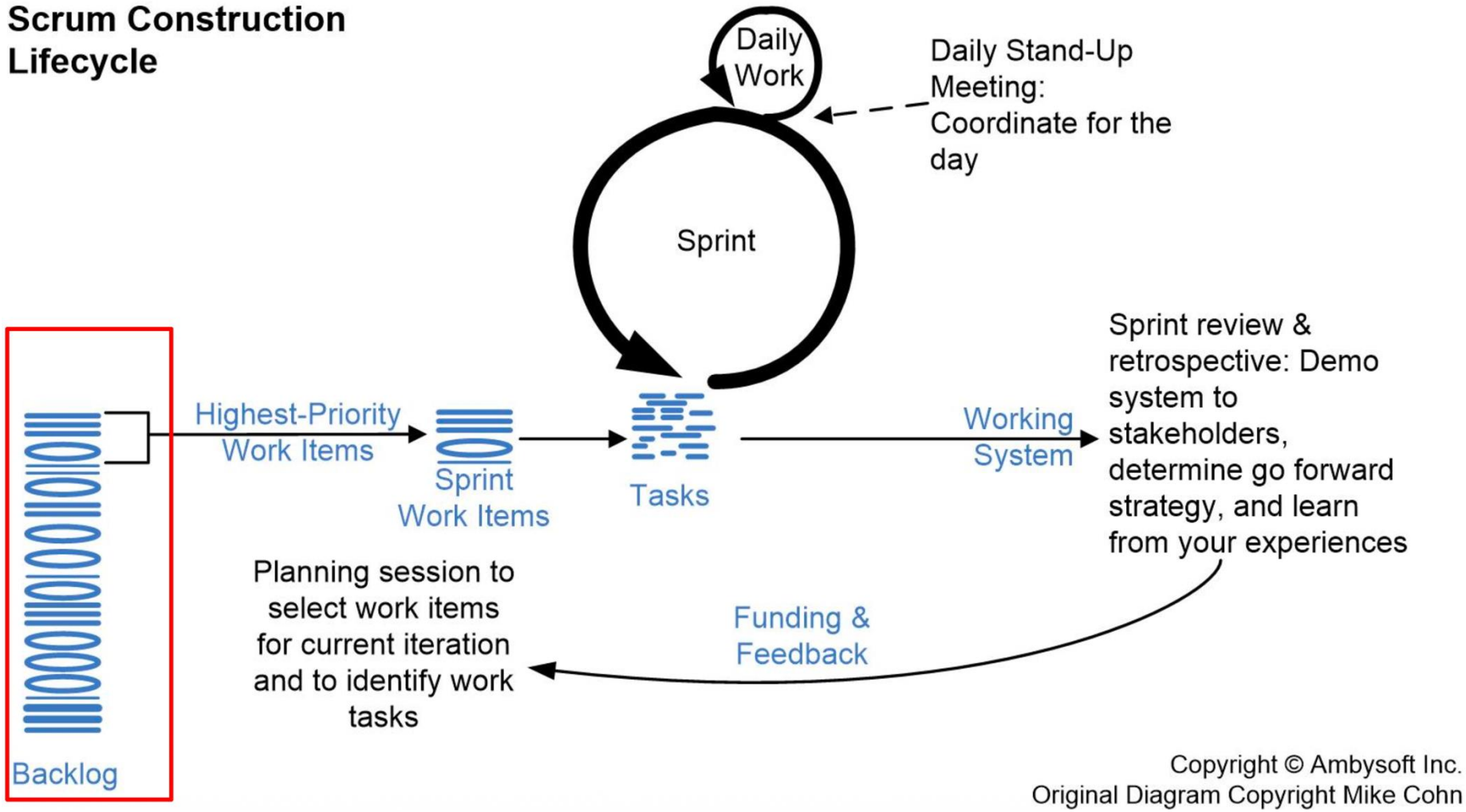
Agile Scrum Requirements



Agile Model Driven Development (AMDD) Requirements



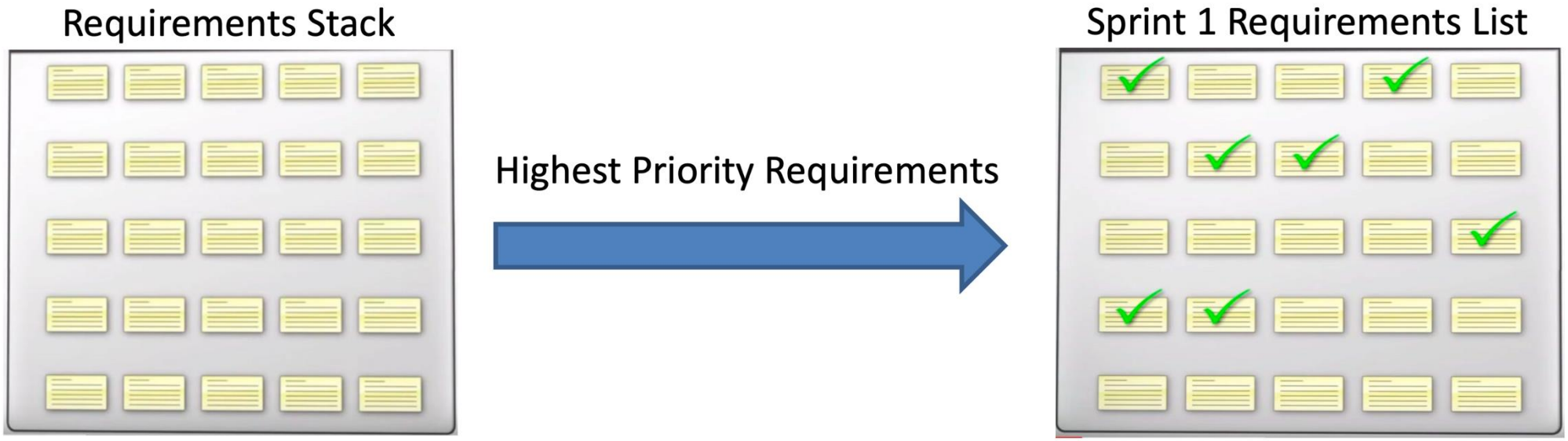
Agile Scrum Construction Lifecyle Requirements



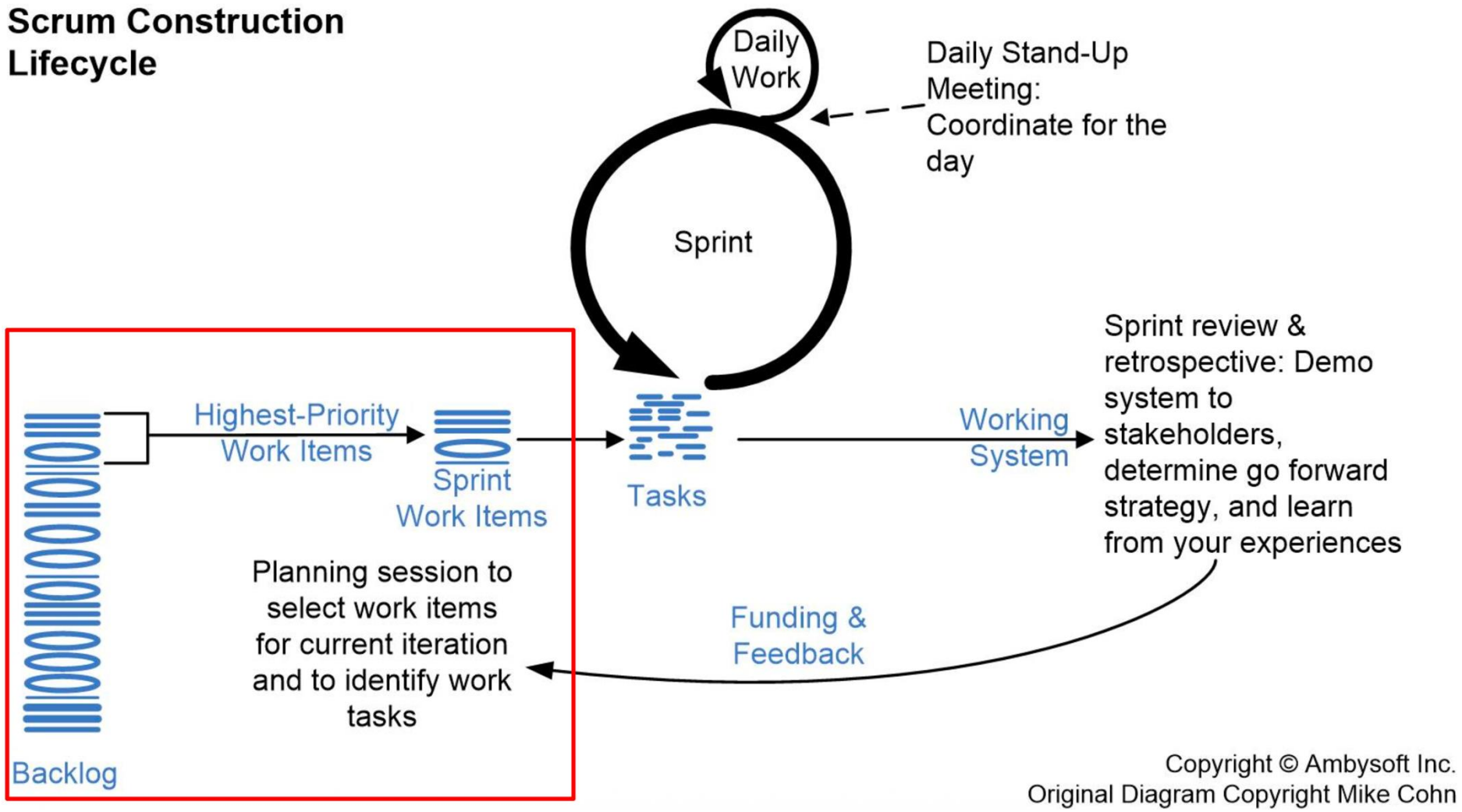
Copyright © Ambysoft Inc.
Original Diagram Copyright Mike Cohn

What Are Requirements Used For

- Prioritized list - priority may change with each sprint
- Highest priority requirements go in first sprint (we'll talk about how many later).



What Are Requirements Used For



What Are Requirements Used For

- After each sprint:
  – New requirements are added to list
  – No longer needed requirements and completed requirements are removed
  – List is re-prioritized
- Highest priority requirements go in second sprint.
- Over and over again until all requirements met or product is retired.

Requirements Engineering Before Agile

- Many traditional project teams run into trouble when they try to define all of the requirements up front.
- The result of a misguided idea that developers will actually read and follow what the requirements document contains.
- The reality is that the requirements document is usually insufficient, regardless of how much effort goes into it.
- The requirements change anyway.
- The developers eventually end up going directly to their stakeholders for information anyway.
- Or they simply guess what their stakeholders meant.

Requirements Engineering with Agile

- If software engineers have the ability to elicit detailed requirements up front, …
- … then they can also do the same when they actually need the information.
- Any investment in detailed documentation early in the project will be wasted when the requirements inevitably change.

Requirements Engineering with Agile

- Fundamental Agile Idea:
- Do just barely enough modeling at the beginning of the project to understand the requirements for your system at a high level.
- Then you gather the details as you need to on a just-in-time (JIT) basis.

Question